

Embedded Firmware Framework Practice

Objective

The objective of this project was to practice modular embedded firmware development using the C programming language. The project simulates a basic embedded firmware architecture commonly used in real microcontroller-based systems.

The implementation focuses on firmware abstraction, modular driver design, runtime state management, and UART-style telemetry output. The project was developed as preparation for embedded systems development and future hardware bring-up tasks.

Modules Implemented

The firmware framework consists of the following modules:

- GPIO Driver
 - GPIO initialization
 - GPIO HIGH/LOW control
 - GPIO toggling
 - GPIO state readback

- UART Driver
 - UART initialization
 - UART message transmission
 - Runtime telemetry output
 - Counter/debug printing
 - Delay Module
 - Software delay implementation
 - Timing simulation for firmware loop execution
 - Board Configuration Layer
 - Board-specific pin definitions
 - Firmware metadata
 - Hardware abstraction
-

Firmware Architecture

The project follows a layered embedded firmware architecture.

Application Layer

↓

GPIO Driver

UART Driver

Delay Driver



Board Configuration Layer

The firmware structure improves:

- code modularity
- scalability
- readability
- maintainability

Features Implemented

The following embedded firmware concepts were implemented:

- Infinite firmware execution loop using while(1)
- Multi-pin GPIO support
- Runtime GPIO state monitoring
- UART-style debug logging
- Counter-based telemetry output
- Modular API-based driver interaction
- Board-level hardware abstraction
- Separate header/source file architecture

Files Included

The project contains the following files:

- main.c
 - gpio.c
 - gpio.h
 - uart.c
 - uart.h
 - delay.c
 - delay.h
 - board.h
-

Concepts Learned

During this project, the following embedded systems concepts were practiced and understood:

- Firmware architecture
- API abstraction
- Modular driver development
- Header and source file separation
- Firmware execution flow
- Runtime debugging

- UART telemetry concepts
 - State management
 - Multi-module firmware organization
 - GCC compilation and linking workflow
-

Program Output

The firmware continuously performs the following operations:

- Toggles multiple GPIO pins
- Reads pin states
- Prints runtime telemetry
- Outputs firmware debug information
- Simulates embedded firmware execution

Example output:

GPIO 13 initialized

GPIO 7 initialized

UART Initialized

VSDSquadron Mini

v1.0

GPIO 13 TOGGLED HIGH

GPIO 7 TOGGLED HIGH

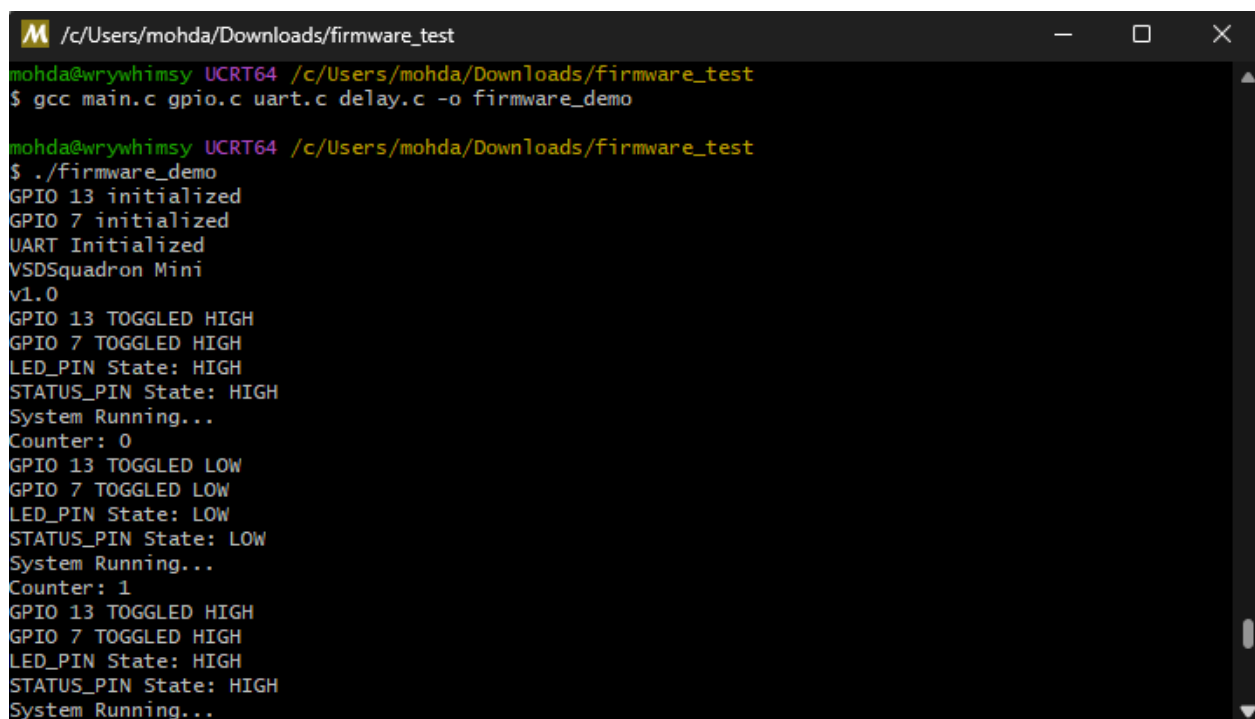
LED_PIN State: HIGH

STATUS_PIN State: HIGH

System Running...

Counter: 0

Output screenshot :



```
M /c/Users/mohda/Downloads/firmware_test
mohda@wrywhimsy UCRT64 /c/Users/mohda/Downloads/firmware_test
$ gcc main.c gpio.c uart.c delay.c -o firmware_demo

mohda@wrywhimsy UCRT64 /c/Users/mohda/Downloads/firmware_test
$ ./firmware_demo
GPIO 13 initialized
GPIO 7 initialized
UART Initialized
VSDSquadron Mini
v1.0
GPIO 13 TOGGLED HIGH
GPIO 7 TOGGLED HIGH
LED_PIN State: HIGH
STATUS_PIN State: HIGH
System Running...
Counter: 0
GPIO 13 TOGGLED LOW
GPIO 7 TOGGLED LOW
LED_PIN State: LOW
STATUS_PIN State: LOW
System Running...
Counter: 1
GPIO 13 TOGGLED HIGH
GPIO 7 TOGGLED HIGH
LED_PIN State: HIGH
STATUS_PIN State: HIGH
System Running...
```

Conclusion

This project helped in understanding the fundamentals of embedded firmware architecture and modular driver development. The implementation simulates how real embedded systems organize hardware abstraction layers, runtime execution loops, GPIO control, and UART telemetry systems.

The project also strengthened understanding of debugging, firmware structure, modular APIs, and scalable embedded software design.
